

Building a Digital Brain for Research:

A Guide to Queryable Knowledge Graphs

Thibault Schrepel¹

April 2026

Forthcoming in the *Network Law Review*, Spring 2026²

Abstract. A digital brain, as coined by Andrej Karpathy, is a personal knowledge infrastructure built from documents you trust. It maps connections between sources, surfaces patterns and inconsistencies. It generates answers on demand with references to the underlying material. The more it is used, the more connections it builds. The output is a private, queryable Wikipedia. On demand, the system generates wiki pages on any theme covered by the corpus, and each new page feeds back into the knowledge base.

This guide documents a pipeline for building such a system from any document corpus, for research purposes. The pipeline adapts Karpathy's methodology and adds six research-specific contributions. (1) A schema design procedure encodes authority hierarchies and surfaces gaps in the literature. (2) A centrality-weighted wiki generation procedure anchors each article around the most-connected sources. (3) A six-step research protocol produces hypotheses rather than retrieved information. (4) Claim-level extraction moves the unit of analysis from documents to propositions, which makes visible the incompatibilities that document-level graphs hide. (5) A persistent hypothesis register stores every query-generated conjecture and re-tests it as the corpus grows. (6) A complexity-theoretic diagnostic layer measures the graph's network properties and reports how the field is structured. The pipeline is field-neutral. The two implementations documented here, one academic research corpus and one European Commission decisions dataset, are illustrative examples.

¹ Vrije Universiteit Amsterdam, Amsterdam Law and Technology Institute;
Stanford CodeX Center for Legal Informatics, Stanford Computational Antitrust Project.

² Any comments are more than welcome. Please feel free to reach out to me at t.schrepel@vu.nl.

Note on attribution

This guide is a technical document that describes how the author assembled existing tools into a research workflow and applied that workflow to two corpora. The knowledge base methodology on which this guide is based was developed by Andrej Karpathy, who described the approach of maintaining a folder of raw documents (papers, notes, screenshots) and using an AI assistant to organize and query them.³ The Graphify skill, which builds the knowledge graph at the center of this pipeline, was developed by Safi Shamsi.⁴ MarkItDown, which converts PDFs to Markdown, was developed by Microsoft.⁵ The scripts for downloading and processing European Commission decisions were written by the author for the purposes described in this guide.

The author's contributions are six in scope. (1) A schema design procedure that the original Graphify methodology does not specify. (2) A centrality-weighted wiki generation procedure that the Node Centrality Weighting rule in Section 4.2 encodes and that Section VII executes. (3) A six-step research protocol that extends the pipeline from a retrieval system into a hypothesis-generation instrument. (4) Claim-level extraction that moves the unit of analysis from documents to propositions. (5) A persistent hypothesis register that stores every query-generated conjecture and re-tests it as the corpus grows. (6) A complexity-theoretic diagnostic layer that measures the graph's network properties and reports what they imply for the field.

All six are adaptations for research purposes, not original technical work. The diagnostic layer assembles existing network-science measurements into a module that Graphify does not produce by default. Still, the primary value of this guide may simply be to flag to researchers that tools of this quality exist. Andrej Karpathy and others have built something that most researchers have not encountered. This guide, if anything, is an attempt to put it on their radar.

³ A Karpathy, described and documented in N Spisak, "How to Build Your Second Brain" (2026), which records the methodology of maintaining a raw/ folder and instructing an AI assistant to compile and maintain a wiki from its contents.

⁴ S Shamsi, Graphify, github.com/safishamsi/graphify (2025).

⁵ Microsoft, MarkItDown, github.com/microsoft/markitdown (2024).

I. Introduction

Large institutions accumulate documents over decades. Decisions, enforcement guidelines, sector inquiries, white papers, and internal reports pile up in filing systems and public registries. Each document was produced in a particular context, often by teams with little visibility over what other teams had written before. The result is a body of material that is rich in substance and opaque in structure. Connections between documents remain hard to grasp. The evolution of analytical positions is also hard to trace. Inconsistencies are difficult to detect.

The same problem confronts researchers. A scholar working at the frontier of any field has read hundreds of papers and cited many of them. Reading a paper and citing it are not the same as retaining its argument. By the time a researcher writes, the connections between sources that seemed clear during reading have often faded, and the older the source, the truer this is. A paper read a decade ago survives in memory as a conclusion at best, rarely as an argument. The knowledge graph addresses this directly. It externalizes the structure of the literature, it makes connections visible and queryable at the moment of writing rather than relying on memory.

This guide presents a pipeline that addresses these problems. It takes a folder of PDF files and produces three things. The first is a clean Markdown version of every document, standardized for AI consumption. The second is a knowledge graph that extracts concepts and reasoning patterns from across the corpus and maps them into a queryable structure. The third is a wiki, i.e., a set of Wikipedia-style articles organized by thematic cluster, with cross-links between related topics and a master index.

The most consequential feature of the resulting system is the ability to query the entire corpus in natural language. Once the graph is built and connected to an AI assistant, anyone with access to the system can ask a question and receive an answer grounded exclusively in the source documents. The AI reads from the corpus. It does not generate content beyond what the documents contain. Every claim it makes is traceable to a specific source.⁶ The pipeline requires no programming expertise. It can be completed in a couple of hours by someone comfortable with a computer.

⁶ This constraint is enforced by the graph structure itself. The AI assistant navigates the graph to locate relevant nodes before generating a response, which limits it to content already extracted from the documents.

II. Overview of the Pipeline

The pipeline has five steps plus ongoing maintenance. Each step feeds into the next. The process can be interrupted and resumed at any point without loss of progress.

- **Step 1 — Collect.** Gather source documents as PDF files. Organize them into thematic subfolders.
- **Step 2 — Convert.** Convert all PDFs to Markdown using MarkItDown, a Python utility developed by Microsoft.⁷ The conversion is automated and parallel.
- **Step 3 — Graph.** Build the knowledge graph using Graphify, an AI coding assistant skill.⁸ Graphify dispatches parallel AI agents that extract concepts and relationships from every document and merge the results into a single graph.
- **Step 4 — Wiki.** Generate the wiki from the knowledge graph. Each thematic cluster in the graph becomes a wiki article, with source summaries and cross-links to related topics.
- **Step 5 — Query.** Ask questions of the corpus through the AI assistant. Use structured graph commands to trace paths between specific nodes, explain concepts, or surface clusters of related decisions.
- **Maintenance.** Drop new documents into the appropriate folder and re-run the pipeline. Only new or modified files are re-processed.⁹

Any step in this pipeline can be clarified by an AI assistant. If a command does not run, if a script returns an unexpected error, or if a step is unclear, take a screenshot of the relevant section of this guide and share it with Claude or another AI system. Alternatively, share the full document and describe what you are trying to do. The guide itself becomes the context for the conversation. There is no need to retype the technical steps or explain the pipeline from scratch.

This approach works at every stage, installation errors, script failures, Graphify commands, wiki generation, and query syntax. The AI reads the relevant section and responds to the specific problem described.

⁷ The MarkItDown repository is available at github.com/microsoft/markitdown. As of April 2026, the project had more than 110,000 stars on GitHub and is distributed under the MIT license.

⁸ The Graphify repository is available at github.com/safishamsi/graphify. As of April 2026, the project had more than 27,000 stars on GitHub and is distributed under the MIT license.

⁹ This is slightly adapted to research purposes from Nick Spisak at https://x.com/NickSpisak_/status/2040448463540830705.

III. Prerequisites

3.1 Hardware

A standard laptop or desktop computer is sufficient. No dedicated graphics processor is required. Eight gigabytes of RAM is sufficient for corpora up to several thousand documents.

3.2 Software

The following tools must be installed before starting. All are free and open-source except Claude Code.

Tool	Install command	License
Python 3.10+	macOS: brew install python – Windows: python.org	PSF
MarkItDown	pip3 install 'markitdown[pdf]' ¹⁰	MIT
Graphify	pip install graphifyy (note double y), then graphify install ¹¹	MIT
Claude Code	docs.anthropic.com/claude-code	Subscription
requests	pip3 install requests	Apache 2.0
Obsidian	obsidian.md	Free (personal use)

Python must be installed on your computer before running any of the scripts in this guide. To check whether it is already installed, open the terminal and type:

```
python3 --version
```

If a version number appears, Python is installed and you can proceed. If not, install it with one command. On macOS:

```
brew install python
```

On Windows, go to python.org, download the installer, and run it. Check the box that says “Add Python to PATH” during installation.

3.3 Subscription and Token Budget

Graphify dispatches parallel AI agents to extract concepts from documents. Each agent consumes API tokens from the AI provider connected to Claude Code. Two subscription tiers are available:

¹⁰ All MarkItDown commands in this guide follow the documentation maintained by Microsoft at github.com/microsoft/markitdown.

¹¹ Graphify commands in this guide follow the documentation maintained by S Shamsi at github.com/safishamsi/graphify. They have been slightly adapted for research purposes, but all credit is due to the S. Shamsi.

Claude Pro (~20 USD/month) is sufficient for corpora up to a few hundred documents. For a corpus of 500 decisions, expect to run two or three sessions as the token limit is reached. Graphify's SHA-256 cache (described in Section VI) means nothing is re-processed between sessions. Re-running the pipeline resumes exactly where the previous session ended.

Claude Max (~100 USD/month) is the recommended option. It elevates rate limits. One month of Claude Max is enough to complete the build, after which the subscription can be reduced. All subsequent updates are incremental and much cheaper.

This guide uses Claude Code and the Anthropic API. The same pipeline works with GPT Codex using Graphify's OpenAI-compatible mode. Other AI systems will support equivalent workflows as they develop.

3.4 Scalability

The pipeline scales with corpus size. The SHA-256 cache is what makes large-scale operation practical: each document is processed once, hashed, and cached. Re-running in a new session resumes from exactly where the previous session ended. No document is processed twice.

Corpus size	Approximate time
20-100 documents	Under 5 minutes. A single session is sufficient.
500 documents	15-30 minutes across one or two sessions.
1,000 documents	30-60 minutes across two or three sessions.
5,000 documents	A few hours spread across a morning.
10,000 documents	One full day at most, again, depending on tokens credit.

IV. Collecting a Research Corpus

This section covers collecting academic papers, reports, decisions, datasets, or other research documents.

4.1 Folder Organization

Create three folders into the main one (your “root folder”):

- raw/ for unprocessed source material that the AI reads but never modifies
- wiki/ for the AI-maintained organized output, and
- outputs/ for generated reports and analyses.

4.2 The CLAUDE.md File

Before running any AI commands, create a file called CLAUDE.md in the root folder. This file tells the AI what the knowledge base is about and how to organize the wiki. A minimal version would be sufficient to begin:

```
# Knowledge Base Schema

## What This Is
A research knowledge base on [YOUR TOPIC].

## How It's Organized
- raw/ contains source documents. Do not modify these files.
- wiki/ contains the organized wiki. The AI maintains this entirely.

## Wiki Rules
- Every topic gets its own .md file in wiki/
- Every wiki file begins with a one-paragraph summary
- Link related topics using [[topic-name]] format
- Maintain an INDEX.md listing every topic with a one-line description
- When new sources are added, update the relevant wiki articles

## Focus Areas
[List 3-5 things you want the knowledge base to focus on]
```

However, the above code has not been designed for research purposes. It does not prescribe how to structure the CLAUDE.md file beyond its role as a general instruction file for the AI assistant. For general software engineering and personal knowledge management applications (the primary use cases for which Graphify was developed) this is sufficient. A minimal schema is enough to produce a useful graph.

For research applications, a more deliberate schema design changes what the system produces. The CLAUDE.md file is the only instrument through which a researcher can shape the graph before it is built. Everything inside it becomes part of the AI's working context during extraction. Used carefully, it determines whether the resulting system retrieves information or generates hypotheses.

Four design choices differentiate a research schema from a minimal one. The first one is especially useful to legal research and can be skipped for another field. I left it here for inspiration. The other three apply across domains.

- **Authority weighting.** Legal and regulatory research corpora typically contain documents with different levels of authority. A corpus that includes both academic commentary and court judgments should encode the hierarchy explicitly, so that the AI draws on each layer appropriately when synthesizing answers. A corpus mixing scholarship, administrative decisions, and appellate judgments requires three tiers. The weighting instruction prevents the AI from treating a student note and a Court of Justice judgment as equivalent sources.
- **Absence as a query target.** Standard schemas instruct the AI to summarize what sources say. A research schema should instruct it to flag what sources do not address. The graph will surface

connected components and suggested questions automatically, but explicit schema instructions amplify this: ask the AI to note, in every wiki article, which adjacent topics the sources in that cluster fail to address.

- **Conceptual anchors.** Research in a defined domain has a vocabulary. Encoding the field's core terms and contested concepts in the schema focuses extraction on what matters to the research question, reducing noise from incidental references. For antitrust research, concepts like market definition, theory of harm, standard of proof, and efficiency defense should appear explicitly. The AI will extract these as priority nodes, which changes the community structure of the resulting graph.
- **Claim-level extraction.** Treating every paper as a single node hides the propositional content inside the paper. Two papers that look connected at the document level often defend incompatible claims at the propositional level. Instruct the AI to extract, for each document, a list of affirmative propositions. Each proposition carries a premise and an inference drawn from that premise, with a pointer to the in-document evidence that supports it. Attach the propositions to the document node as child nodes. The six-step query protocol then runs at claim granularity. An absent path between two claims carries different meaning than an absent path between two documents.

A research schema for a legal corpus combining scholarship and regulatory decisions might look as follows (the part highlighted in blue is for legal researchers only; the part in yellow to be changed by all users):

Research Knowledge Base Schema

What This Is

A research knowledge base on [TOPIC]. The corpus combines academic scholarship, regulatory texts, administrative decisions, and court judgments. These have different authority levels and should be treated accordingly.

How It's Organized

- raw/ contains source documents. Do not modify these files.
- wiki/ contains the organized wiki. The AI maintains this entirely.

Wiki Rules

- Every topic gets its own .md file in wiki/
- Every wiki file begins with a one-paragraph summary
- Link related topics using [[topic-name]] format
- Maintain an INDEX.md listing every topic with a one-line description
- When new sources are added, update the relevant wiki articles

Focus Areas

[List 3-5 things you want the knowledge base to focus on]

Source Authority Hierarchy

1. Court judgments (highest authority; treat as binding)

2. Administrative decisions (authoritative; treat as primary sources)
3. Regulatory texts and guidelines
4. Peer-reviewed scholarship
5. Working papers and commentary (lowest authority)

When sources at different levels address the same point, cite the higher authority and note how it confirms, modifies, or overrides the lower-authority source.

Node Centrality Weighting

The knowledge graph ranks documents by how many connections they have to other documents. Documents with the most connections are god nodes and reflect the field's consensus more than isolated documents. When generating wiki articles and synthesizing answers, give more weight to god nodes than to documents with few connections. If no court judgments are yet in the corpus, apply this rule to the available sources: the most-cited and most-connected documents carry more weight regardless of document type.

Wiki Rules

- Every topic gets its own .md file in wiki/
- Every wiki file's summary is built primarily from the god nodes in that cluster, not from all sources equally
- Every wiki file includes a section titled "What the sources do not address" identifying adjacent questions the corpus leaves open
- Link related topics using [[topic-name]] format
- Maintain an INDEX.md listing every topic with a one-line description

Core Concepts (extract these as priority nodes)

[List 8-12 field-specific terms that are central to the research question]

Research Focus

[Describe the 2-3 questions the knowledge base should be positioned to answer]

Claim Extraction Rules

For each document, extract the list of affirmative propositions it contains. Each proposition carries a premise and an inference drawn from that premise, with a pointer to the in-document evidence that supports the inference. Attach each proposition to its document node as a child node. The proposition nodes are queryable alongside the concept nodes.

Hypothesis Register

Every query that surfaces an absent path, a disconnected cluster, a god node, or a concept mentioned but never defined produces a hypothesis. Append each hypothesis to hypotheses.md with the date, the query that generated it, the graph state at the time, and the evidence the query returned. When the corpus is updated, re-test every stored hypothesis against the new graph and append the result to the register as a dated entry.

Diagnostic Module

After each graph build, compute the graph's network properties and report them in GRAPH_REPORT.md. The degree distribution tells you whether the graph follows a power law. The clustering

coefficient together with the mean path length tells you whether the graph qualifies as small-world. The entropy across communities measures whether attention is concentrated in a few clusters or spread evenly. A year-over-year trace of these metrics reveals whether the field is consolidating or diversifying. When more than one graph is available, compare the metrics across graphs and report each divergence.

The “What the sources do not address” instruction directs the AI to look for silences rather than content. In a field structured by debates, the unanswered questions are often more useful than the answered ones.

This schema design step is distinct from anything in the original Graphify documentation. It represents an adaptation of the pipeline specifically for research purposes. The claim is not that schema design was impossible before (any user can put whatever they like in CLAUDE.md) but that a deliberate research schema design step, treated as a required part of the pipeline rather than an optional configuration, changes the character of the output from a retrieval system to a research instrument.

4.3 Downloading the files

You need to download all the file that will be used to train your digital brain and put them into your folder “raw.” You can download them manually or automate the process should it be permitted by the institution holding them.

In the case of the European Commission, for example, the institution publishes all competition case data in a single downloadable JSON file through the EU Open Data Portal. One download, one script execution, and the corpus is assembled. In what follows, I explain how I executed the script to download EC decisions, a step you can skip should you want to do it manually:

1. Go to the EU Open Data Portal at:
data.europa.eu/data/datasets/18489cb7-bce7-4d44-a138-795b390d2109--1?locale=en
2. Download the case data file in JSON format. The filename is case-data-AT.json.
3. Place the file in your folder, for example /Users/yourname/BRAIN EC/.

Open a text editor, create a new file, and save it as “ec_decisions_pipeline.py” in the same folder as case-data-AT.json. The script reads the JSON file, extracts all English PDF links and downloads them. Run this code in your Terminal:

```
#ec_decisions_pipeline.py

import json, logging, re, time
from pathlib import Path
from concurrent.futures import ThreadPoolExecutor, as_completed
import requests

JSON_FILE = Path('case-data-AT.json')
```

```

OUTPUT_DIR = Path('decisions')
PDF_DIR = OUTPUT_DIR / 'pdfs'
WORKERS = 6
SKIP_DONE = True
DELAY = 0.5
TIMEOUT = 60

PDF_DIR.mkdir(parents=True, exist_ok=True)
logging.basicConfig(filename=str(OUTPUT_DIR / 'skipped.log'),
    level=logging.ERROR, format='%(asctime)s %(message)s')
HEADERS = {'User-Agent': 'Mozilla/5.0 (compatible; research-bot/1.0)'}

def safe_filename(case_id, title, date, seq):
    slug = re.sub(r'[\w\s-]', '', title or 'untitled')
    slug = re.sub(r's+', '-', slug.strip())[:60]
    suffix = f'_{seq}' if seq > 0 else ''
    return f'{case_id}_{(date or "0000-00-00")[:10]}_{slug}{suffix}'

def extract_decisions(json_path):
    with open(json_path, encoding='utf-8') as f:
        data = json.load(f)
    records = []
    for case_id, case in data.items():
        meta = case.get('metadata', {})
        title = (meta.get('caseTitle') or '')[0]
        case_type = (meta.get('caseCartel') or '')[0]
        sectors = meta.get('caseSectors', [])
        companies = meta.get('caseCompanies', [])
        for dec in case.get('decisions', []):
            dec_meta = dec.get('metadata', {})
            dec_date = (dec_meta.get('decisionAdoptionDate') or '')[0]
            dec_types = dec_meta.get('decisionTypes', [])
            for seq, att in enumerate(dec.get('decisionAttachments', [])):
                att_meta = att.get('metadata', {})
                lang = (att_meta.get('attachmentLanguage') or '')[0]
                links = att_meta.get('attachmentLink') or []
                if not links or lang != 'EN':
                    continue
                records.append({
                    'case_id': case_id, 'title': title, 'type': case_type,
                    'date': dec_date, 'dec_types': dec_types,
                    'sectors': sectors, 'companies': companies,
                    'url': links[0],
                    'filename': safe_filename(case_id, title, dec_date, seq)
                })
    return records

def download_pdf(record):
    pdf_path = PDF_DIR / f'{record["filename"]}.pdf'
    if SKIP_DONE and pdf_path.exists() and pdf_path.stat().st_size > 1000:
        return record, True, 'skipped'
    try:
        time.sleep(DELAY)
        r = requests.get(record['url'], headers=HEADERS,
            timeout=TIMEOUT, stream=True)
        r.raise_for_status()

```

```

pdf_path.write_bytes(r.content)
return record, True, 'ok'
except Exception as e:
    logging.error(f'DOWNLOAD {record['case_id']}: {e}')
    return record, False, str(e)

def run_phase(label, records, fn, workers):
    total = len(records)
    done = skipped = failed = 0
    with ThreadPoolExecutor(max_workers=workers) as pool:
        futures = {pool.submit(fn, r): r for r in records}
        for i, future in enumerate(as_completed(futures), 1):
            _, ok, status = future.result()
            if status == 'skipped': skipped += 1
            elif ok: done += 1
            else: failed += 1
            bar = █ * int(30*i/total) + ░ * (30-int(30*i/total))
            print(f'\r{label} [{bar}] {i}/{total} '
                  f'done={done} skipped={skipped} failed={failed}',
                  end=' ', flush=True)
    print(f'\n -> {done} done, {skipped} skipped, {failed} failed\n')

def main():
    records = extract_decisions(JSON_FILE)
    print(f'Found {len(records)} English decision PDFs')
    run_phase('Download', records, download_pdf, WORKERS)
    print(f'Done. PDFs saved to {PDF_DIR}/')

if __name__ == '__main__':
    main()

```

Then run this one into your Terminal:

```
pip3 install requests
```

4.4. Renaming the files

Once you have all the files; you want to rename them. Renaming them takes one of two forms. The first is manual. For a corpus of 1 to 50 files this is manageable. For 50 or more it is not. The second is automated. Give Claude Cowork access to the folder and state the naming convention you want applied. For example:

Rename every file in this folder using the format: [Case number] – [Case name].pdf. Derive the case name from the existing filename. Keep it under 40 characters.

The AI reads the existing filenames, applies the rule, and renames the files without opening or modifying their contents.

V. Converting PDFs to Markdown

This step applies to the research corpus pipeline.

5.1 Why Markdown

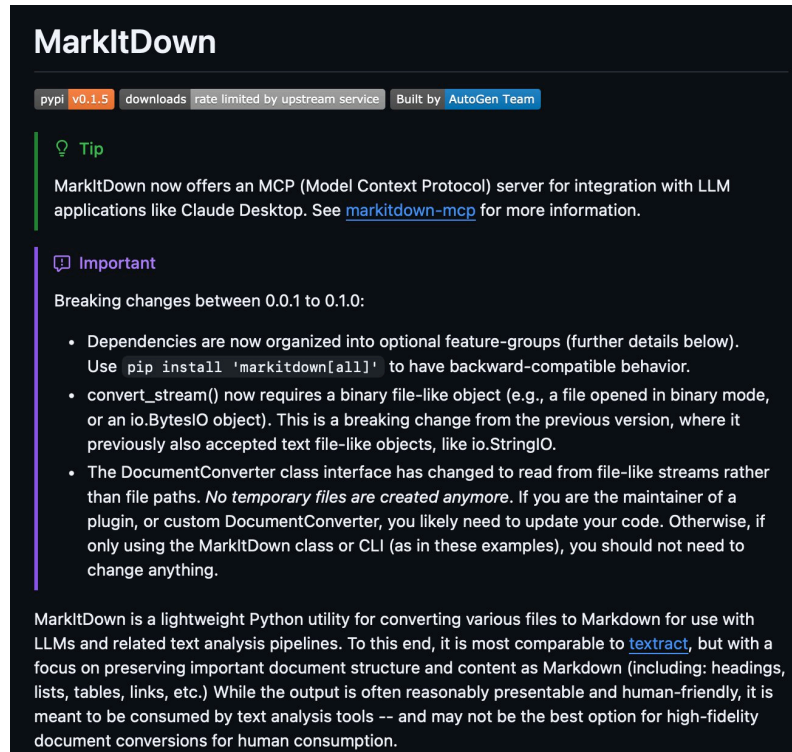


Figure 1. MarkItDown GitHub repository (microsoft/markitdown). Over 110,000 stars, MIT license.

The PDFs downloaded in the previous step cannot go directly into the knowledge graph. They first need to be converted to plain text otherwise the AI reads layout noise rather than content. The processing of PDFs also consumes more tokens. Markdown preserves document structure (headings, lists, tables...) while removing layout information (fonts, page breaks, column positions) that adds no semantic value. MarkItDown, developed by Microsoft, is designed specifically for this conversion.¹² It supports PDF, Word, PowerPoint, Excel, HTML, and more.

5.2 The Batch Conversion Script

Open a text editor, create a new file, and save it as “batch_convert.py” in your working directory. Before running it, open the file and **replace the two placeholder paths on lines 6 and 7** with the actual paths to your PDF folder and your desired output folder.

¹² See <https://github.com/microsoft/markitdown>.

```

#batch_convert.py
import logging
from pathlib import Path
from concurrent.futures import ThreadPoolExecutor, as_completed
from markdown import Markdown

INPUT_DIR = Path('/path/to/your/pdfs') #<- change this
OUTPUT_DIR = Path('/path/to/output') #<- change this
WORKERS = 4
SKIP_DONE = True

logging.basicConfig(filename='conversion_errors.log',
    level=logging.ERROR, format='%(asctime)s %(message)s')

def convert_one(pdf_path):
    relative = pdf_path.relative_to(INPUT_DIR)
    out_path = OUTPUT_DIR / relative.with_suffix('.md')
    out_path.parent.mkdir(parents=True, exist_ok=True)
    if SKIP_DONE and out_path.exists():
        return pdf_path, True, 'skipped'
    try:
        result = Markdown(enable_plugins=False).convert(str(pdf_path))
        out_path.write_text(result.text_content, encoding='utf-8')
        return pdf_path, True, 'ok'
    except Exception as e:
        logging.error(f'{pdf_path}: {e}')
        return pdf_path, False, str(e)

def main():
    pdfs = sorted(INPUT_DIR.rglob('*.pdf'))
    total = len(pdfs)
    if total == 0:
        print('No PDFs found.')
        return
    done = skipped = failed = 0
    with ThreadPoolExecutor(max_workers=WORKERS) as pool:
        futures = {pool.submit(convert_one, p): p for p in pdfs}
        for i, future in enumerate(as_completed(futures), 1):
            _, ok, status = future.result()
            if status == 'skipped': skipped += 1
            elif ok: done += 1
            else: failed += 1
            bar = '█' * int(30*i/total) + '░' * (30-int(30*i/total))
            print(f'\r[{bar}] {i}/{total} '
                f'done={done} skipped={skipped} failed={failed}',
                end='', flush=True)
    print(f'\nDone. {done} converted, {skipped} skipped, {failed} failed')

if __name__ == '__main__':
    main()
pip3 install 'markdown[pdf]'
python3 batch_convert.py [Markdown command syntax follows Microsoft's documentation at
github.com/microsoft/markitdown.]

```

Open the terminal, navigate to the folder where you saved the file, and run the following commands. Replace `/Users/yourname/BRAIN` with the actual path to your working directory (your root folder):

```
# cd /Users/yourname/BRAIN
pip3 install 'markitdown[pdf]'
python3 batch_convert.py
```

The script mirrors the original folder structure in the output directory. `SKIP_DONE = True` means it is safe to re-run after any interruption.

VI. Building the Knowledge Graph

6.1 What Graphify Does

Graphify reads a folder of files and produces a knowledge graph. It runs inside Claude Code.

Graphify runs in three passes. A deterministic AST pass extracts structure from code files without using any AI. A semantic pass dispatches parallel AI agents, up to 31 simultaneously. Each agent extracts concepts and relationships from its assigned documents and returns a structured JSON payload. The payloads are merged into a NetworkX graph, clustered using Leiden community detection, and exported in three formats: interactive HTML, queryable JSON, and an audit report in Markdown.

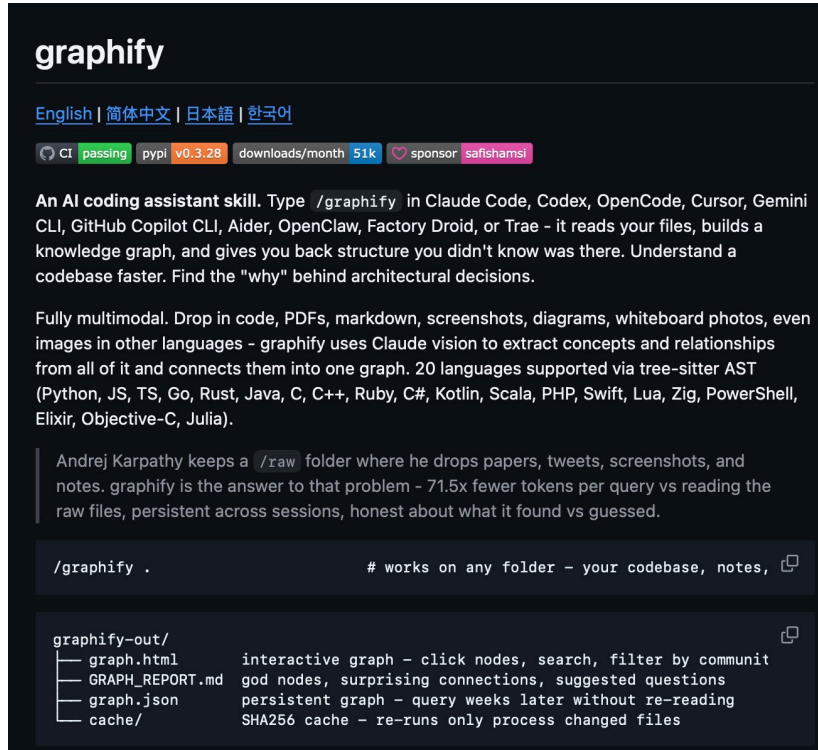


Figure 2. Graphify GitHub repository (safishamsi/graphify). Over 27,000 stars, MIT license.

6.2 Installing Graphify

```
pip install graphify # note double y
graphify install # registers the skill in Claude Code^[Graphify command syntax follows Safi Shamsi's
documentation at github.com/safishamsi/graphify.]
```

6.3 Running on a Large Corpus

For a corpus of 674 files (what I had for the first digital brain I created with research papers), Graphify warns that the corpus is too large to process in a single pass. Type all to run the full pipeline across all files. If the session token limit is reached before extraction completes, open a new session and re-run the same command. The SHA-256 cache ensures that files already processed in the previous session are skipped:

```
/graphify /path/to/markdown/files
# When prompted: all

# In subsequent sessions if interrupted:
/graphify /path/to/markdown/files --update
```


6.4 Results on the Research Corpus

```

all

Running the full pipeline on all 674 files. This will take a while — dispatching semantic extraction
subagents in parallel.

> Exécuté 3 commandes

Semantic extraction: ~674 files → 31 agents, estimated ~3-5 min (all running in parallel).

> Exécuté 2 commandes

Dispatching all 31 extraction subagents in parallel now.

v Extract knowledge graph chunk 2

Bash $(cat graphify-out/.graphify_python) -c " import json from pathlib import Path chunks = json.loa...
736 /tmp/graphify_all_chunks.txt

Agent Extract knowledge graph chunk 1
Async agent launched successfully.
agentId: a4f06f2ec27b3ee59 (internal ID - do not mention to user. Use SendMessage with to: 'a4f06f2ec27
b3ee59' to continue this agent.)
The agent is working in the background. You will be notified automatically when it completes.

Agent Extract knowledge graph chunk 2

```

Figure 3. Graphify running the full pipeline across 674 files, dispatching 31 extraction subagents in parallel.

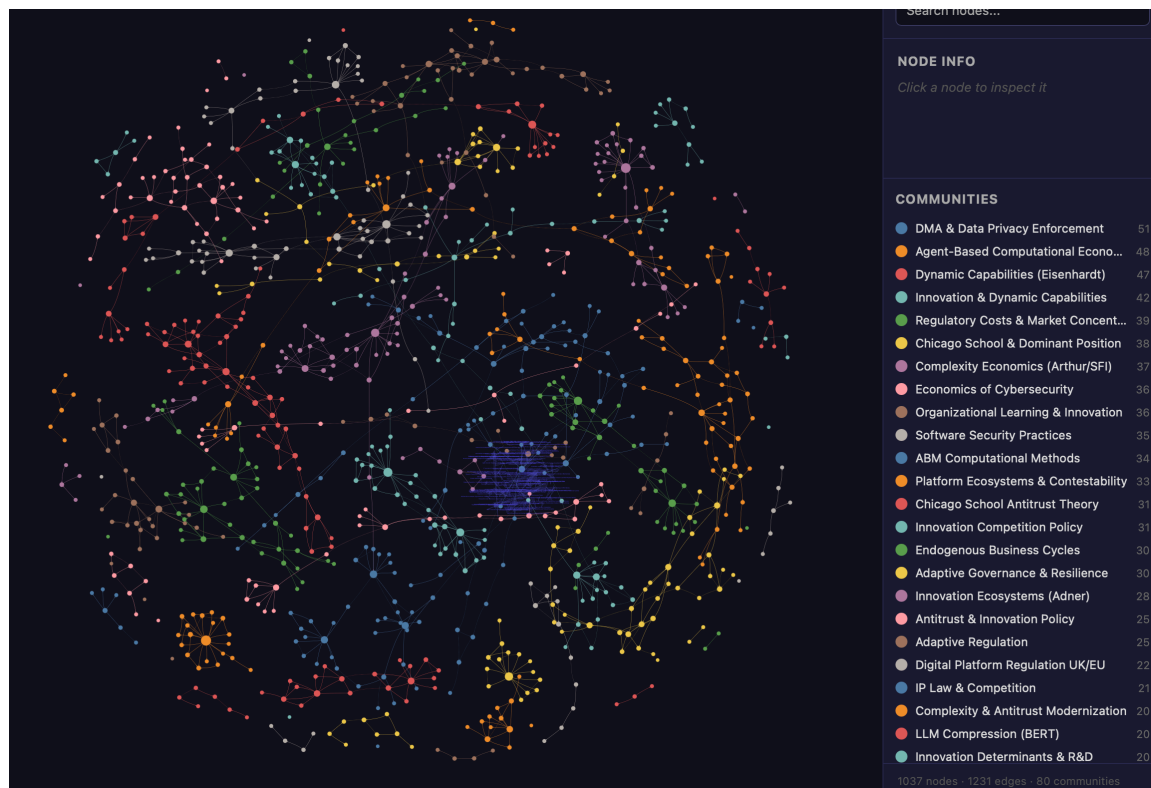


Figure 4. The interactive knowledge graph (graph.html). Each node is a concept, case, or framework. Colors represent thematic communities.

Just to give an example, on the 674-file corpus, Graphify produced the following:

Metric	Value
Source files	674 documents, approximately 9.5 million words
Raw tokens	approximately 12.7 million
Extraction tokens	approximately 56,000 (225-fold compression)
Nodes	1,037
Edges	1,231
Communities	80
Processing time	3-5 minutes with 31 parallel agents

VII. Generating the Wiki

This section implements the centrality-weighted wiki generation procedure. The rule lives in the CLAUDE.md schema as the Node Centrality Weighting block in Section 4.2. The wiki generation step reads the rule and applies it. Each wiki article draws its summary primarily from the god nodes in its thematic cluster. Sources with few connections contribute less. The schema supplies the instruction. Section VII carries it out.

7.1. Via Claude Code Prompt

Run “/graphify -wiki” in Claude Code first. If it returns an error, use the prompt below instead.

Open Claude Code, point it at the Markdown folder, and type the following. Replace the path with the actual location of your Markdown files.

Read everything in XXXX. Then compile a wiki in wiki/ following the rules in CLAUDE.md. Create an INDEX.md first, then create one .md file per major topic. Link related topics. Summarize every source.

XXXX should be uploaded with the file directory where you put all your markdown files, such as:
/Users/yourname/BRAIN/Files MD

This approach dispatches parallel read agents and produces the same wiki structure. The centrality weighting and authority hierarchy defined in CLAUDE.md apply automatically. The AI reads that file before generating the wiki and follows its instructions. Expect the process to take a couple minutes depending on corpus size.

7.2. Opening in Obsidian

1. Open Obsidian.
2. Choose Open folder as vault.

3. Select the wiki/ folder.
4. Click INDEX.md to begin navigating.

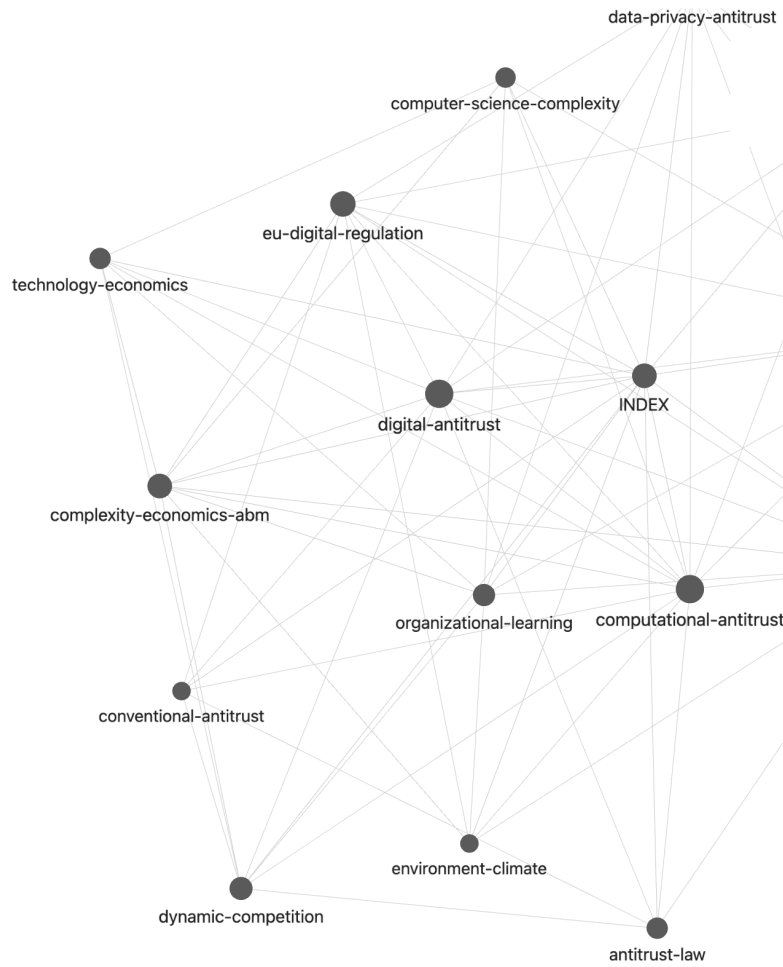


Figure 5. The Obsidian graph view of the wiki. Each node is a topic page. Edges represent cross-links. The layout mirrors the knowledge graph community structure.

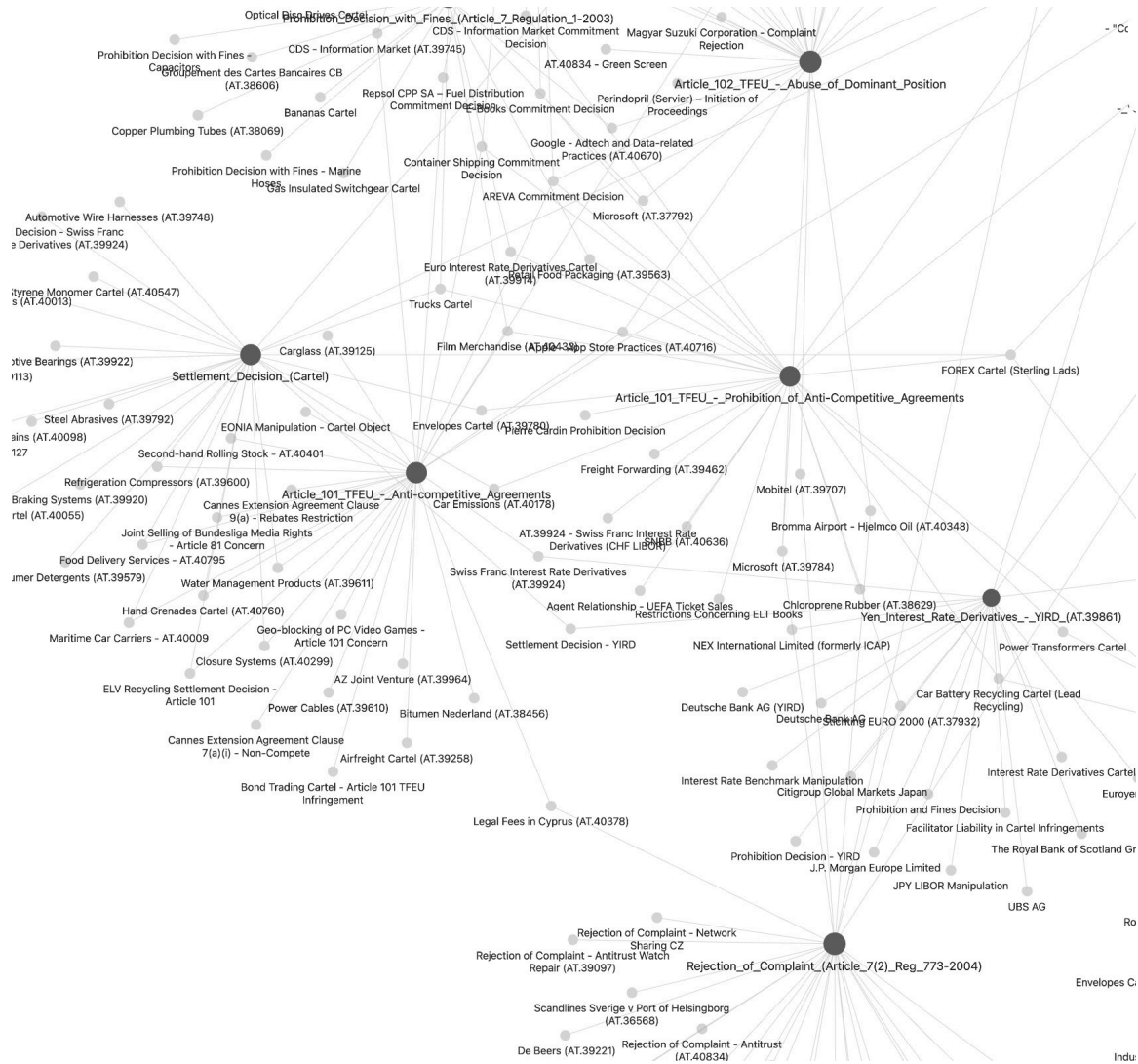


Figure 6. The EC competition decisions knowledge graph. Central nodes correspond to Articles 101 and 102 TFEU. Peripheral nodes are individual decisions.

Obsidian renders the cross-links between pages as clickable hyperlinks and provides a graph view showing how all topic pages relate to each other. The graph view is navigable. Clicking any node opens the corresponding wiki page.

VIII. Querying and Exploring

8.1 Enable Always-On Integration

Run the following once in the terminal:

```
graphify claude install
```

This installs a hook that instructs Claude Code to read the graph report before answering any question. Claude navigates by graph structure rather than by keyword search. Every answer is grounded in the corpus. The graph report identifies the most central documents and the surprising connections between them, and Claude draws on these when generating responses. For best results, begin each session by asking Claude to confirm it has loaded the graph before posing substantive questions.

8.2 Query Commands

You made it. You can now, within Claude Code, start querying the Wiki by asking any question you may have. The knowledge graph finds documents that are structurally connected to a concept, even if they never use that exact term. In my example, it reveals cases that share the same underlying economic logic without addressing the same market. It traces how a doctrinal position has shifted across a chain of related decisions. It identifies structurally isolated clusters (decisions that share no extracted connections with the rest of the corpus) which may represent emerging enforcement areas or doctrinal dead ends.

The god nodes (the highest-degree concepts in the graph) are the foundational documents around which the rest of the corpus is organized. Their centrality is not imposed by a human editor. It emerges from the pattern of citations and conceptual links across the entire corpus.

This is reinforced by the Node Centrality Weighting instruction in CLAUDE.md, which explicitly tells the AI to give more weight to the most-connected documents when synthesizing answers or, in the case of legal research, the give more weight to Court of Appeals judgments. The hook and the schema work together. The hook ensures the graph is loaded, and the schema determines how it is used.

Every query leads to the creation of a new wiki page that is automatically integrated within the graph. This means that the more you use the brain, the smarter it becomes because it keeps on making new connections.

digital-antitrust

Digital Antitrust

74 source files | Platform markets, algorithmic competition, AI and antitrust, zero-price markets

Related: [Antitrust Law](#) · [Conventional Antitrust](#) · [Computational Antitrust](#) · [EU Digital Regulation](#) · [Data Privacy & Antitrust](#) · [Dynamic Competition](#)

Overview

Digital antitrust grapples with how standard competition law tools apply (or fail) in digital markets: zero-price platforms, multi-sided markets, algorithmic collusion, data-driven market power, and ecosystem competition. The cluster bridges theoretical IO, empirical economics, and doctrinal law reform.

Figure 7. A wiki page for Digital Antitrust in Obsidian, showing 74 source files, key topics, and cross-links to related pages.

Ecosystem Analysis in EC Digital Cases

Research note synthesized from flagship digital platform decisions (2004–2025)

Summary Finding

The Commission demonstrates **emerging but inconsistently applied ecosystem thinking**. It uses "ecosystem" as a contextual frame and a qualifying condition for abuse, but has not developed a unified ecosystem theory of harm. Analysis remains predominantly product-by-product, with the ecosystem treated as background that amplifies standard tying, leveraging, and data-foreclosure theories. Remedies are narrow — interoperability and access orders — never structural ecosystem dismantlement.

Where the Commission Uses "Ecosystem" Explicitly

Apple Mobile Payments (AT.40452, 2024) — Most Ecosystem-Centric Decision

The most developed ecosystem reasoning in the corpus. The Commission explicitly states:

"When considering (digital) ecosystems, the Commission may apply similar principles to those applied to after-markets to define relevant product markets."

It characterises iOS as a "vertically integrated ecosystem" and treats consumer lock-in as foundational to the abuse:

"iOS users are locked into the ecosystem: consumers can neither switch to an alternative mobile wallet provider, nor reasonably switch to an alternative smart mobile device."

And on the theory of harm:

Figure 8. A wiki page for Ecosystems in EC Digital Cases in Obsidian

IX. A Research Protocol: From Retrieval to Hypothesis Generation

The Karpathy methodology and the Graphify skill are designed primarily for information retrieval. Take a corpus, build a structured system, find documents and concepts efficiently. This improves on keyword search for large corpora. For research purposes, however, retrieval is a starting point rather than an end point. What follows describes a protocol for using the completed knowledge graph to generate research hypotheses rather than locate existing knowledge.

This protocol does not modify the pipeline. It specifies how to query the graph once it is built. The six steps below are designed to be run in sequence on any completed corpus. Together they produce a structured research agenda, a map of what the field has established, where its frontiers lie, and which connections it has failed to make.

9.1. The Six-Step Query Protocol

Step 1 — Identify the god nodes. The god nodes are the highest-degree concepts in the graph, the documents and ideas to which the largest number of other nodes connect. The god nodes identified here determine how the wiki weights its sources. Running this query first and reviewing the output takes five minutes and materially improves the quality of the generated articles. They reveal what the field treats as foundational. In my case, a corpus in which Gundlach and Foer (2006) is the most connected node is a corpus that has organized itself around a particular conception of the relationship between complexity, networks, and antitrust. A corpus in which Intel (AT.37990) is the most connected node is a decisional corpus organized around a particular theory of exclusionary conduct.

```
/graphify query "what are the most connected nodes in the graph"
```

Step 2 — Identify the disconnected components. The knowledge graph will almost always contain clusters of documents with no extracted connections to the main body of the corpus. These are the field's frontiers and its dead ends. A disconnected cluster may represent an emerging subfield that has not yet been cited into the mainstream, a line of argument that was tried and abandoned, or a body of work that the field simply has not yet encountered. Each disconnected component is a research question, that is, why is this material isolated, and what would it take to connect it?

```
/graphify query "identify disconnected components"
```

Step 3 — Query for absent paths. This is one of the most productive steps for generating research hypotheses. Once the god nodes and disconnected components are identified, query for paths between pairs of nodes that should, on theoretical grounds, be connected but are not. The absence of a path between two concepts is a finding. It means the literature has not yet made a connection that theory suggests should exist.

```
/graphify query "identify absent paths in the graph"
```

If the graph returns no path (as it did, in the research corpus, between the Collingridge Dilemma and Arthur's technology lock-in theory) the result is a gap in the literature. No document in the corpus explicitly connects the two. That is a paper potentially waiting to be written.

Step 4 — Cross-corpus divergence (where two graphs exist). Build a second graph on a different corpus covering the same domain and compare the two. A scholarly literature graph and a regulatory decisions graph on the same subject will typically contain different god nodes, different connections. Concepts densely connected in the scholarly graph but absent or isolated in the regulatory graph are candidates for regulatory adoption. Concepts prominent in regulatory decisions but untheorized in scholarship are candidates for academic work. This divergence analysis is most productive when the two corpora are deliberately chosen to represent different institutional perspectives on the same problem.

Step 5 — Authority-chain conflict check (legal corpora). For corpora that include texts at different authority levels (scholarship, administrative decisions, appellate judgments) query for nodes where different authority levels state contradictory positions. A concept that scholarship addresses one way but that a court judgment resolves differently is a site of doctrinal evolution. The graph surfaces these conflicts automatically when the authority hierarchy is encoded in the schema.

Step 6 — The health check. Query for concepts mentioned across the corpus but never explained. These are nodes with many incoming edges (many documents reference them), but no outgoing edges and no wiki article that defines them. They are the field's assumed knowledge. Terms that every document uses but none defines. In legal and regulatory research, these often correspond to foundational concepts whose content is contested precisely because no one has defined them carefully.

```
/graphify query "concepts mentioned frequently but never defined"
```

9.2 The Absence-First Orientation

The protocol above is organized around the principle that in mature research fields, the most useful finding is a gap. Content retrieval (finding what the corpus says about a topic) is readily available through conventional search. What is not available through conventional search is a structural map of what the corpus does not say. The knowledge graph produces this map as a byproduct of extraction. Querying for absent paths, disconnected components, and undefined concepts turns that byproduct into a primary output.

The "What the sources do not address" section in each wiki article is the visible surface of this orientation. It directs every wiki article to end with a statement of what the documents in that cluster fail to address. Over a corpus of several hundred documents, these statements accumulate into a research agenda.

9.3 The Institutional Memory Audit

For corpora built from an institution's own decisional output (in my case, competition decisions, judicial opinions, regulatory guidance) the six-step protocol produces a specific type of output that differs from its application to a scholarly corpus. The god nodes reveal which decisions the institution's own case law has treated as foundational, not by any human editorial judgment, but by the pattern of citations and conceptual dependencies that the institution itself has created over time. The disconnected components reveal doctrinal orphans: decisions that were adopted but never followed up, theories of harm that were articulated but never developed, markets that were investigated once and never revisited.

This output is an institutional memory audit. It tells an institution what its accumulated decisions have established as central, where its enforcement record contains gaps, and which lines of reasoning were started and left incomplete. For an antitrust agency, the audit identifies which decisions the next case handler should read before drafting, which market definitions have never been tested on appeal, and which theories of harm appear in the decisional record but have no supporting line of decisions behind them.

9.4 The Hypothesis Register

The six-step protocol produces a set of hypotheses each time it runs. An absent path is a conjecture. A god node is a structural claim about the field. A disconnected cluster is a prediction that a body of work has not been integrated. A concept mentioned but never defined is a claim about the field's assumed knowledge.

The hypothesis register stores each of these as a dated entry in `hypotheses.md`. Every entry records the generating query. It also stores the graph state at the time and the evidence the query returned. When the corpus is updated, the register re-runs each stored query against the new graph. The result is appended to the original entry with a new date.

The register accumulates into a research diary that documents the fate of each conjecture as the corpus grows. A hypothesis that held in April 2026 and failed in April 2027 is a documented shift in the field. A hypothesis that held across two years of new material is a proposition the field has not contradicted. Both outcomes are research findings.

The implementation is one instruction in `CLAUDE.md` and a standing command to re-test on update.

9.5 Complexity-Theoretic Diagnostics

The knowledge graph is a network. Networks can be measured against the benchmarks that network science has established. A research pipeline that operates on a graph should report those measurements as part of its output. The diagnostic module runs after each graph build and writes its results into `GRAPH_REPORT.md`.

The degree distribution tells you whether the graph is scale-free. A scale-free graph has a small number of god nodes that accumulate a disproportionate share of the connections and a long tail of nodes with few connections each. Most mature scholarly fields are scale-free. A corpus that produces a flat degree distribution is one whose attention has not found its centers, which usually means the extraction has missed them.

The clustering coefficient together with the mean path length tells you whether the graph is small-world. A small-world graph is one in which most nodes can be reached from any other in a few steps and in which connections cluster locally. Most research fields satisfy both conditions. A graph that fails the small-world test is a graph whose subcommunities are barely linked, and the failure is itself a finding.

The entropy across communities measures whether attention is concentrated or diffuse. A field organised around a small number of dominant frames has low community entropy. A field in broad exploration has high entropy. The trend of this metric across successive graph builds reveals whether the field is consolidating or expanding.

When more than one graph is available, the diagnostics run pairwise. A scholarly graph and a regulatory graph on the same domain will typically diverge on these metrics. Each divergence is a diagnostic signal about the gap between the two institutional perspectives on the problem.

The module is one query run against the NetworkX graph that Graphify already builds. The results take a few seconds to produce and a few lines to report.

9.6 What This Protocol Adds

The Karpathy methodology, implemented through Graphify and MarkItDown, produces a queryable knowledge base. The protocol described in this section adds a defined procedure for extracting research value from that knowledge base that goes beyond retrieval. The schema design shapes the graph before it is built. The six-step query protocol mines the graph for findings after it is built. The hypothesis register preserves each finding as a dated entry and re-tests it as the corpus grows. The diagnostic module measures the graph's own network properties and reports what they imply for the field. Together they constitute an approach to using these tools for research that is distinct from the general-purpose applications for which they were developed.

X. Keeping It Updated

A knowledge graph that goes stale loses its value. The pipeline is designed so that keeping it current requires minimal effort. Graphify's SHA-256 cache ensures that only new or changed files are re-processed. The existing graph is not rebuilt from scratch. New nodes and edges are merged into the existing structure, communities are reclustered, and the wiki is regenerated.

Adding a single new document. In the terminal:

```
# Convert the PDF
markdown new_decision.pdf -o /path/to/markdown/new_decision.md
```

Then in Claude Code:

```
# Merge into the existing graph
/graphify /path/to/markdown/files --update

# Regenerate the wiki
/graphify /path/to/markdown/files --wiki
```

Continuous monitoring. Run the following in the terminal to rebuild the graph automatically whenever files change:

```
# Rebuild the graph automatically whenever files change
/graphify /path/to/markdown/files --watch

# Rebuild after every Git commit and branch switch
graphify hook install
```

XI. Outputs

After completing the pipeline, the following files are available:

File	Description
graph.html	Interactive visualization. Open in any browser. Click nodes, search, filter by community. No server required.
graph.json	Full graph in JSON format. Queryable programmatically or through Claude Code commands. GraphRAG-ready.
GRAPH_REPORT.md	Audit report containing the god nodes, the most surprising connections, and four to five suggested research questions the graph is positioned to answer. Also reports the network-science diagnostics produced after each build, covering the degree distribution, the clustering coefficient, the mean path length, and the entropy across communities.
hypotheses.md	Persistent register of the hypotheses generated by each six-step protocol run. Each entry records the generating query together with the graph state and the evidence returned at the time. Re-tested automatically on each corpus update.
wiki/INDEX.md	Master index listing every topic page with a one-line description.

File	Description
wiki/*.md	Topic pages covering all thematic communities, with per-source summaries, key concepts, cross-cutting themes, and bidirectional cross-links.
graphify-out/cache/	SHA-256 cache of all processed files. Enables incremental updates.
decisions/skipped.log	EC pipeline only. Log of failed downloads or conversions, with timestamps and error messages.

XII. Tools Reference

Tool	Details
MarkItDown	github.com/microsoft/markitdown – pip3 install 'markitdown[pdf]' – MIT license
Graphify	github.com/safishamsi/graphify – pip install graphify – MIT license
Claude Code	docs.anthropic.com/claude-code – Claude Pro, Team, or Max; also works with GPT-4o
Obsidian	obsidian.md – Free for personal use
Python 3.10+	python.org – macOS: brew install python
EU Open Data Portal	data.europa.eu – EC competition case data in machine-readable JSON with direct PDF links
EUR-Lex	eur-lex.europa.eu – Pre-2000 decisions not in the EC open data file; CJEU and General Court judgments

XIII. Additional Use Cases

The use cases described below are drawn from my field of research. They are illustrative, not exhaustive. Every discipline will generate its own applications once the system is built.

Decision consistency for agencies and courts

An administrative agency drafting a new decision can drop the draft into a digital brain built from all previous decisions and query it for consistency. The system will surface prior decisions that address the same legal standard, the same theory of harm, or the same market definition. Inconsistencies become visible before the decision is issued rather than on appeal. The same applies to courts: a draft judgment can be checked against a corpus of prior rulings to identify conflicts with established doctrine and departures from the rule of law that would otherwise require correction at a higher instance.

Legal practice and peer review

Practicing lawyers can use the same approach. A brief can be checked against a corpus of decisions for doctrinal consistency. An economic theory can be tested against the literature before it is submitted. More broadly, the system enables a researcher or practitioner to drop a document into an existing brain and interact with it directly, rather than relying on memory of what has been read in the past. A paper read ten years ago survives in memory as a conclusion; in the brain, it survives as an argument with traceable connections. This has direct application to peer review as well, subject to the journal or editor permitting the use of AI for that purpose.

Team and institutional deployment

A digital brain does not have to be private. Three configurations are available. The first is personal use: a single researcher maintains a private brain built from sources they trust. The second is team use: a shared folder where multiple collaborators deposit new sources, and the brain grows with every addition. The third is institutional deployment: a court, a regulator, or a research institution publishes a queryable version of its own decisional or scholarly corpus on its website, allowing anyone to query it in natural language without requiring technical knowledge of the underlying structure. A competition authority could offer this for its published decisions. A court could offer it for its judgments. A journal could offer it for its archive. The pipeline described in this guide supports all three configurations without modification.

Regulatory mapping for policymakers

The two implementations documented in this guide (a research corpus and a body of competition decisions) share a common architecture with a third application that policymakers should consider building, i.e., a digital brain covering the full regulatory corpus of a given domain.

The EU digital regulatory framework illustrates why this matters. As of 2024, eight major regulations govern the digital economy: the Data Governance Act, the Digital Markets Act, the Digital Services Act, the Digital Operational Resilience Act, the Chips Act, the Data Act, the Artificial Intelligence Act, and the Cyber Resilience Act. These instruments were adopted over a three-year window (2022-2024) by different legislative teams responding to different perceived problems. They define the same concepts differently, impose overlapping obligations on the same regulated entities, and leave structural gaps in coverage that are visible only when the texts are read together. No single person has read all eight acts with sufficient attention to map every overlap and every gap. The acts together run to several hundred pages, with hundreds of delegated and implementing acts to follow.

A knowledge graph built from these eight regulations would make this structure visible. It would identify, for example, every provision in which the term “AI system” is used and compare the definitions in force across acts. It would surface every article that grants the Commission a monitoring obligation and reveal where monitoring obligations conflict or duplicate each other. It would trace which regulated entities are subject to obligations under multiple acts simultaneously and what those obligations require of them. It would identify concepts that appear in one act but are not addressed in adjacent acts despite evident relevance.

The query commands apply directly. A policymaker drafting a new delegated act under the AI Act could ask the graph to surface every provision across all eight regulations that touches on the same subject matter. A parliamentary committee reviewing the DMA could ask the graph to trace all regulatory obligations that currently apply to gatekeeper platforms and identify where those obligations conflict with obligations under the DSA or the Data Act.

The same pipeline produces all of this. The corpus changes (regulations replace competition decisions) but the technical steps are identical: convert PDFs to Markdown, build the graph with Graphify, generate the wiki, and query the result. The time required to build such a system from scratch, assuming the regulatory texts are already available in PDF form, is a couple of days at most. A policymaker who completes this once has a system that updates automatically as new acts are adopted or amended.

The scale of EU digital regulation is not exceptional. Many regulatory domains (financial services, pharmaceutical approval, telecommunications, data protection) have accumulated comparable or larger bodies of overlapping rules. The pipeline works on any of them.¹³

¹³ For an analysis of the adaptive properties of the EU Digital Acts specifically, including the degree to which they embed mechanisms for monitoring and revision, see T Schrepel, “Adaptive Regulation” (2026) *European Journal of Risk Regulation*, doi:10.1017/err.2026.10087.

Appendix A – Computational Antitrust: EC Investigative and Analytical Methods

Research note synthesized across 508 EC competition decisions (1977–2025)

Summary

The Commission’s enforcement toolkit remains predominantly qualitative and documentary. References to computational or econometric methods in decisions are sparse, typically reactive – evaluating parties’ submissions rather than deploying Commission-generated models – and never describe a proactive algorithmic detection infrastructure. There is no visible computational antitrust methodology in the decisional record.

What the Commission Does Reference

Traffic and click-through data analysis. In Google Shopping (AT.39740, 2017), the most data-intensive factual record in the corpus, the Commission analysed click-through rates and user behaviour patterns from Google’s own systems. The Commission compared volumes and trends before and after the conduct in question. It did not apply an econometric model. The analysis is documentary: compare traffic before and after, note the decline.

Cost accounting and data allocation. In Aspen (AT.40394, 2021), faced with incomplete cost data, the Commission “used a volume-based allocation methodology” (recital 1608) to estimate regional costs. This is data imputation. It is not causal inference.

The Merchant Interchange Test (Visa/MasterCard). In Visa (AT.39398) and MasterCard (AT.40049), the Commission applied the MIT formula using market investigation parameters. This is a regulatory calculation formula – deterministic arithmetic, not statistical modelling.

External consumer surveys. In Apple Mobile Payments (AT.40452, 2024), the Commission cites survey data (footnote 55: 90% of iOS users prefer iPhones). These are surveys conducted by third parties, not by the Commission. No statistical methodology, margin of error, or inferential framework is disclosed.

Critiquing parties’ econometrics. In Intel (AT.37990, 2009), the Commission’s most sustained engagement with statistical methodology appears as a critique. Recitals 1065 to 1081 detail methodological flaws in Intel’s regression analysis: omitted variable bias, specification errors, and data limitations. The Commission’s own assessment remained qualitative. The pattern recurs throughout the corpus: parties submit econometric models, the Commission critiques them, and decisions resolve on documentary grounds. In Qualcomm Predation (AT.39711, 2019), the Commission conducted what the decision itself describes as “a particularly cumbersome analysis of a large amount of quantitative evidence” including R&D cost databases and sales data (recitals 1046–1049). The analysis was document review and cost accounting, not regression analysis or causal identification.

What Is Absent

Across 508 decisions spanning 1977 to 2025, there is no mention of: cartel screening algorithms or statistical price-monitoring tools; machine learning or AI for bid-rigging detection; algorithmic collusion detection infrastructure; dawn raid digital forensics platforms or e-discovery tools; the Commission's own proprietary databases or real-time monitoring systems; natural experiments, difference-in-differences, or event study methodologies deployed by the Commission itself; GUPPI or diversion ratio analysis; merger simulation models (these appear in merger decisions outside this corpus).

Algorithms as the Object of Investigation, Not the Investigative Tool

The Commission discusses algorithms – as the subject matter of cases, not as investigative methods.

In the FOREX Cartel cases (AT.40135), traders' use of electronic chatrooms and automated execution is part of the factual record. The Commission's investigation relied on chat transcripts, not algorithmic analysis of trading patterns. In Google Ads (AT.40670, 2025), the Commission analyses how Google's ad auction algorithms function and how they create competitive advantages – using Google's own internal documents and econometric submissions from third parties to establish this, not independent algorithmic analysis. In Amazon Buy Box (AT.40703), the Buy Box algorithm's role in self-preferencing is central to the investigation, but the Commission's analysis rests on Amazon's internal documentation and affected sellers' testimony.

How the Commission Actually Works

The Commission's competition enforcement rests on five methods, none computational in the data-science sense:

Method	Role	Example
Leniency applications	Cartel detection trigger	All major cartel cases
Dawn raids and requests for information	Document collection	All cases
Cost accounting	Predation and margin squeeze analysis	Qualcomm, Aspen
Comparative fact-finding	Traffic, price, and market share comparisons	Google Shopping
Third-party econometrics	Adversarial expert evidence	Intel, E-books

The Commission acts as an evaluator of econometric evidence, not a generator of it. The legal standard – standard of proof in administrative proceedings reviewed by the CJEU – creates an

institutional incentive toward documentary evidence that withstands appeal, rather than statistical modelling that requires methodological disclosure and is vulnerable to adversarial challenge.

Why This Gap Exists

Cartel detection may use computational tools – leniency markers, sector inquiry screens – that predate formal proceedings and therefore never appear in published decisions. The decisional record captures the Commission’s analytical methodology after detection, not its detection infrastructure. DG COMP’s Chief Economist Team does produce economic analysis using GUPPI, diversion ratios, and price concentration studies. This work feeds into internal assessments rather than appearing as formal decisional methodology. The gap between Chief Economist Team analysis and published decisions is structural, not incidental.

Emerging Signals

Google Adtech (AT.40670, 2025) is the most technically sophisticated decision in the corpus. The Commission analyses programmatic ad auction mechanics, real-time bidding dynamics, and data integration across the ad stack at a level of technical granularity absent from earlier decisions. It does not disclose computational methods, but the analysis implies access to expert modelling of auction outcomes. Google AI (AT.40983, 2025) investigates AI training data and model architecture. If the Commission’s analytical methodology is to keep pace with the conduct it investigates, computational tools will become unavoidable. This case may signal where the gap begins to close.

Conclusion

The Commission is investigating increasingly computational conduct – algorithmic pricing, AI-driven market power, programmatic auctions – with a toolkit that remains fundamentally pre-digital. The decisional record contains no evidence that this gap has been closed internally. Whether the Commission’s unpublished investigative infrastructure is more computationally sophisticated than the published decisions suggest is impossible to assess from the decisions alone.

Appendix B – Ecosystem Analysis in EC Digital Cases

Research note synthesized from flagship digital platform decisions (2004–2025)

Summary

The Commission demonstrates emerging but inconsistently applied ecosystem thinking. It uses “ecosystem” as a contextual frame and a qualifying condition for abuse, but has not developed a unified ecosystem theory of harm. Analysis remains predominantly product-by-product. Remedies are narrow – interoperability and access orders – and no remedy attacks the ecosystem structure itself.

Where the Commission Uses Ecosystem Reasoning Explicitly

Apple Mobile Payments (AT.40452, 2024) is the most ecosystem-centric decision in the corpus. The Commission explicitly states that when considering digital ecosystems, it may apply principles similar to those applied to after-markets in defining relevant product markets. It characterises iOS as a vertically integrated ecosystem and treats consumer lock-in as foundational to the abuse: iOS users are locked into the ecosystem and can neither switch to an alternative mobile wallet provider nor reasonably switch to an alternative smart mobile device.

This is the only decision where ecosystem lock-in drives market definition, not only the theory of harm. The after-market analogy is significant. It means the primary market – iOS devices – can confer dominance in adjacent secondary markets – NFC mobile wallets – through ecosystem lock-in, without a separate dominance finding in those adjacent markets.

Google Android (AT.40099, 2018) contains frequent references to developing the Android ecosystem and to GMS devices as the competitive unit. The Commission recognises that Google Play Services’ strength is reinforced by the quantity and popularity of apps available on the Play Store – a network-effect description of ecosystem advantage. The analysis fragments into separate tying, bundling, and revenue-share counts rather than a unified ecosystem theory.

Microsoft Teams II (AT.40873, 2025) describes Microsoft as offering an entire ecosystem of business application software aiming to be a one-stop-shop for organisations’ software needs. The ecosystem framing characterises Microsoft’s strategy. The abuse theory remains tying – Teams bundled with Office 365 – not ecosystem-level harm.

Traditional Framing Dominates Multi-Product Cases

Case	Ecosystem Conduct	Legal Framing
Google Android (2018)	Mandatory GMS bundle, revenue share to block rival apps	Tying and exclusive dealing
Google Shopping (2017)	Search self-preferencing	Abuse of dominance under Article 102(c)

Case	Ecosystem Conduct	Legal Framing
Facebook Marketplace (AT.40684, 2021)	Social network data leveraged to advantage Marketplace	Tying and data advantage
Microsoft (2004)	Windows dominance leveraged to servers and media	Cross-market network effect leveraging
Google Adtech (AT.40670, 2025)	Ad server, SSP, and DSP integration advantages	Abuse via integration, not ecosystem theory

In each case, the ecosystem is visible as a pattern across counts but is not the unit of legal analysis.

Data Flows Across the Portfolio

Google Adtech (AT.40670, 2025) offers the most granular treatment. The Commission analyses holistic data integration across the Google stack and the ability to de-duplicate data from across Google services, allowing a more accurate picture of advertising effectiveness. Data cross-subsidy across the ad tech stack is presented as an abuse-enabling advantage, not as a standalone ecosystem harm theory.

No decision in the corpus theorises a data flywheel or ecosystem data loop as a self-contained theory of harm. Data advantages appear in the dominance section as barriers and in the abuse section as enabling conditions. They do not appear as standalone ecosystem theories.

Theories of Harm: Fragmented, Not Unified

The Commission has not asked, in any single decision: does this firm's ecosystem as a whole foreclose competition? The question it consistently asks is: does this specific conduct in this specific market foreclose competitors in that market?

The harm theories it applies in ecosystem-adjacent cases are: tying and bundling across products, as in Android and Teams; leveraging dominance from a primary market to an adjacent market, as in Google Shopping and Microsoft; after-market abuse through ecosystem lock-in, as in Apple NFC; foreclosure through data advantage, as in Facebook and Google Adtech; and self-preferencing, as in Google Shopping and Amazon Buy Box. Each count is assessed individually.

Remedies

Case	Remedy
Apple NFC (2024)	Must grant effective NFC access – iOS ecosystem remains intact
Google Android (2018)	Restrictions on anti-fragmentation obligations – GMS bundle survives
Microsoft (2004)	Unbundling of Windows Media Player – weakly enforced
Google Adtech (2025)	Access and interoperability obligations – ad stack integration survives

The Commission has never ordered structural separation of a digital ecosystem. Its consistent preference for behavioural remedies – access orders, interoperability requirements, non-discrimination conditions – implicitly accepts that integrated ecosystems are legitimate as long as specific abusive conduct within them is corrected.

The After-Market Analogy

The Apple Mobile Payments decision suggests a doctrinal path toward ecosystem-level analysis. Treating the primary ecosystem platform as analogous to an aftermarket-creating product would allow a single dominance finding (in iOS) to generate liability across multiple adjacent markets without separate dominance proof in each, and would allow ecosystem lock-in to serve as the theory of harm rather than requiring product-specific foreclosure analysis in each affected market.

Whether this becomes the standard approach in post-DMA Article 102 enforcement is uncertain. The DMA's gatekeeper designation framework already operates on essentially this logic – designating the core platform service and then regulating conduct across all adjacent services – but Article 102 enforcement has not yet adopted it systematically.

Conclusion

The Commission sits in a transitional position. Ecosystem advantages are visible in every major digital decision. They are legally operationalised only partially. In Apple NFC, ecosystem logic drives market definition. In Google Android and Google Adtech, ecosystem framing appears in the background while individual abuse counts remain product-by-product. Until the Commission either extends the Apple after-market analogy broadly or pursues structural remedies, ecosystem thinking remains context rather than doctrine.